Edge AI Chess Engine : QAT, Distillation et Optimisation Bas Niveau sur Architecture ARM 32-bits

Ajyad HASSANI
Élève-Ingénieur (1A) – Télécom SudParis
Rôle : Algorithmique & Apprentissage Profond

Mars 2026

Table des matières

1 Résumé Exécutif **1**

2 Introduction **1**

2.1 Sélection de la plateforme : STM32F411 1

2.2 Capacité de calcul et architecture 1

2.3 Objectifs de l'EDD 1

3 Architecture du Moteur d'Évaluation (NNUE) **1**

3.1 Représentation de l'échiquier (One-Hot Encoding) 2

3.2 Formalisme de l'Accumulateur 2

3.3 Démonstration de la Mise à Jour Incrémentale 2

3.4 Gain de complexité et impact sur la mémoire Flash 3

1 Résumé Exécutif

Ce document détaille la conception d'un moteur d'échecs hybride sur **STM32F411**. L'innovation majeure réside dans l'intégration d'un réseau de neurones de type **NNUE** (*Efficiently Updatable Neural Network*) comme fonction d'évaluation, optimisé pour une exécution sans unité de calcul flottant (FPU).

Afin de respecter la « loi d'airain » du matériel, le modèle suit une chaîne de transformation rigoureuse :

— **Architecture Frugale** : Une couche cachée limitée à 32 neurones pour minimiser l'empreinte mémoire.

— **Quantification Int16** : Une conversion intégrale en arithmétique entière 16 bits. Ce choix garantit une exécution déterministe et performante sur un processeur non optimisé pour les calculs tensoriels.

— **Contraintes Matérielles** : Le système est dimensionné pour opérer dans une enveloppe stricte de **128 Ko de RAM** et **512 Ko de Flash**.

2 Introduction

2.1 Sélection de la plateforme : STM32F411

Le STM32F411 a été retenu pour son rapport performance/consommation optimal en contexte *Edge*. Bien que doté d'un cœur Cortex-M4, ses limitations intrinsèques dictent des choix algorithmiques orientés vers l'efficacité brute.

2.2 Capacité de calcul et architecture

Le système repose sur un processeur **ARM Cortex-M4 32 bits** monocœur cadencé à **100 MHz**.

— **Gestion du flux** : L'absence de multi-cœur impose un partage strict du temps processeur entre la recherche arborescente (Minimax) et l'inférence du réseau. L'optimisation du code est donc vitale au maintien de la profondeur de recherche.

— **Arithmétique Entière** : Bien que le cœur dispose d'une FPU simple précision, l'inférence neuronale en *float* est proscrite au profit d'une stratégie **Int16**. Cette approche exploite la puissance native du jeu d'instructions 32 bits pour transformer l'inférence en une séquence d'opérations entières à haute vélocité.

2.3 Objectifs de l'EDD

Le présent document vise à démontrer que l'alliance d'un réseau de neurones et d'une quantification Int16 permet d'atteindre une précision d'évaluation supérieure aux heuristiques classiques, tout en respectant les limites thermiques et énergétiques d'un système embarqué.

3 Architecture du Moteur d'Évaluation (NNUE)

Le choix d'un réseau de neurones de type **NNUE** répond à la nécessité de dépasser les limites de précision des fonctions d'évaluation statiques traditionnelles tout en respectant les contraintes de latence d'un système embarqué. Cette architecture, composée d'une unique couche cachée

couplée à une fonction d'activation de type **CReLu** (*Clipped ReLU*), repose sur la capacité du réseau à subir des mises à jour différentielles.

3.1 Représentation de l'échiquier (One-Hot Encoding)

L'état physique de l'échiquier est projeté dans un espace vectoriel de dimension $N = 768$. Cet espace permet un mappage bijectif entre une configuration locale {pièce, couleur, case} et un index unique k. L'indexation est régie par la fonction de hachage suivante :

$$k = \text{pièce} + (6 \times \text{couleur}) + (12 \times \text{case}) \tag{1}$$

Les variables de cette fonction sont discrétisées selon les plages suivantes :

- **pièce** $\in \{0, 1, 2, 3, 4, 5\}$ correspondant respectivement à $\{P, N, B, R, Q, K\}$.
- **couleur** $\in \{0, 1\}$ où 0 représente les Blancs et 1 les Noirs.
- **case** $\in \{0, 1, \ldots, 63\}$ pour les 64 cases de l'échiquier.

Le vecteur d'entrée est défini comme un vecteur ligne binaire $\mathbf{x} \in \{0, 1\}^{1\times 768}$. Par construction, $\mathbf{x}$ est une représentation *One-Hot* : pour chaque case occupée, une seule composante parmi les 12 possibles est active (valeur 1), les autres étant nulles (0).

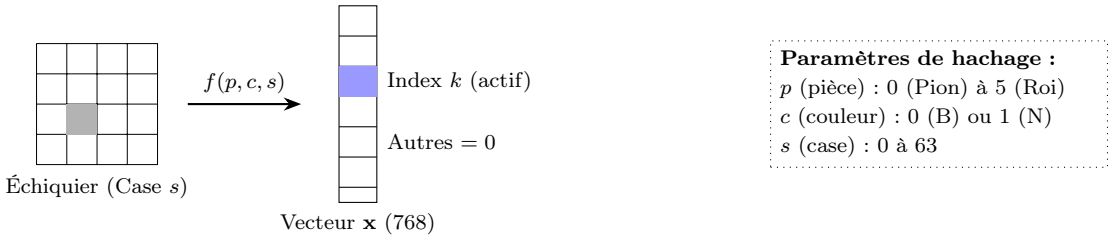

FIGURE 1 – Représentation One-Hot Encoding et projection vers l'index k.

3.2 Formalisme de l'Accumulateur

La première couche du réseau agit comme un accumulateur de caractéristiques. Pour garantir une implémentation logicielle directe sur architecture 32 bits, nous définissons l'accumulateur $\mathbf{A}$ comme un vecteur ligne de dimension 32.

Soit $\mathbf{W} \in \mathbb{Z}^{768\times 32}$ la matrice des poids synaptiques. Nous définissons $\mathbf{w}_k \in \mathbb{Z}^{1\times 32}$ comme le **vecteur ligne** correspondant à la k-ième ligne de la matrice $\mathbf{W}$. L'état de l'accumulateur est défini par :

$$\mathbf{A} = \mathbf{x}\mathbf{W} + \mathbf{b} = \left(\sum_{k, x_k = 1} \mathbf{w}_k \right) + \mathbf{b} \tag{2}$$

Où $\mathbf{b} \in \mathbb{Z}^{1\times 32}$ est le biais. L'accumulateur correspond mathématiquement à la somme vectorielle des poids des caractéristiques actives.

3.3 Démonstration de la Mise à Jour Incrémentale

La performance du moteur sur **STM32F411** repose sur l'exploitation de la corrélation entre deux positions successives. Lors d'un mouvement déplaçant une pièce d'une case source (s) vers une case destination (d), le vecteur $\mathbf{x}$ ne subit que deux modifications : l'indice i (pièce sortante) passe à 0 et l'indice j (pièce entrante) passe à 1.

Preuve par substitution : Soit $\mathbf{A}_t$ l'état de l'accumulateur à l'instant t. Par définition de la forme sommatoire :

$$\mathbf{A}_t = \left(\sum_{k \neq i,j, x_k = 1} \mathbf{w}_k \right) + \mathbf{w}_i + \mathbf{b} \tag{3}$$

L'état suivant $\mathbf{A}_{t+1}$ s'écrit en désactivant l'indice i et en activant l'indice j :

$$\mathbf{A}_{t+1} = \left(\sum_{k \neq i,j, x_k = 1} \mathbf{w}_k \right) + \mathbf{w}_j + \mathbf{b} \tag{4}$$

En isolant le terme de l'état précédent dans l'équation, nous identifions directement la relation de récurrence :

$$\mathbf{A}_{t+1} = \underbrace{\left(\sum_{k \neq i,j, x_k = 1} \mathbf{w}_k + \mathbf{w}_i + \mathbf{b} \right)}_{\mathbf{A}_t} - \mathbf{w}_i + \mathbf{w}_j \tag{5}$$

D'où la relation fondamentale : $\mathbf{A}_{t+1} = \mathbf{A}_t - \mathbf{w}_i + \mathbf{w}_j$.

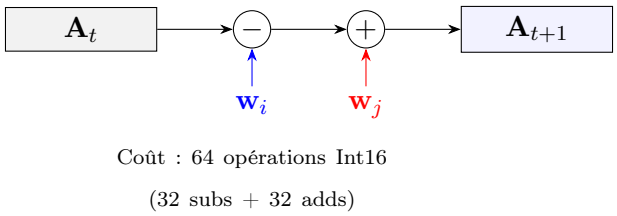

FIGURE 2 – Mécanisme de mise à jour différentielle de l'accumulateur.

3.4 Gain de complexité et impact sur la mémoire Flash

Cette démonstration prouve que l'inférence est réduite à une complexité constante $O(32)$. Le coût CPU par nœud est limité à **64 opérations entières**. Sur le plan matériel, ce choix est validé par son empreinte sur la Flash (512 Ko) :

— **Stockage de la matrice W** : $768 \times 32 \times 2$ octets (Int16) $= 49\,152$ octets.

— **Occupation relative** : L'accumulateur n'occupe que **9,6 % de la Flash totale**.